

LearningSteps Evolution: From Manual to Automated

DevOps / DevSecOps capstone | Oleh Rushchak | Cybersteps, Sept-Oct 2026 | github.com/olehrush/learningsteps

1. Goal and result

The LearningSteps API (FastAPI + PostgreSQL), previously deployed by hand on Azure VMs, was rebuilt as an automated, self-healing and observable system. Every **git push** now lints and tests the code, builds a Docker image, scans code, image and Terraform, pushes the verified image to Azure Container Registry and rolls it out to AKS. All three mandatory success criteria and the optional monitoring criterion were demonstrated.

Request path: Browser → Azure Load Balancer (public IP) → Service → 2-4 API Pods (HPA) → PostgreSQL Flexible Server (private subnet, TLS). Pods read the DB connection string from Key Vault via Workload Identity. **Release path:** push → GitHub Actions (OIDC, no stored Azure secret) → test → build → scan → ACR → AKS rolling update.

2. Implementation

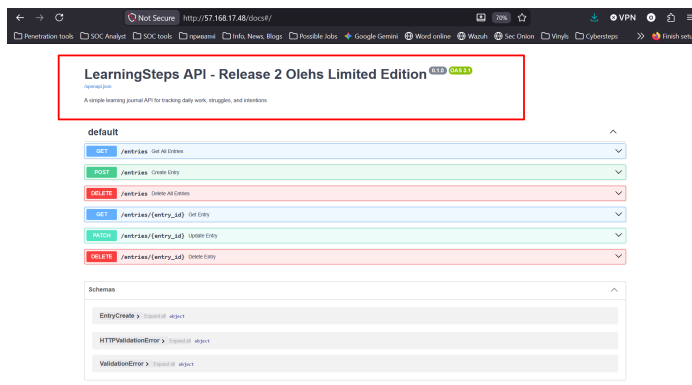
Area	What was built	Security decisions
Container	Python 3.12 slim image, non-root user, container test against a real PostgreSQL before push	Minimal base, non-root, unique tag per commit (no :latest)
IaC (Terraform)	Resource group, VNet with AKS + delegated DB subnets, AKS (2x D4s_v6), ACR, PostgreSQL Flexible, Key Vault, managed identities, RBAC - 29 resources; remote state in a separate locked-down Storage account	Private DB, TLS enforced, KV deny-by-default + IP allowlist, AKS local accounts disabled, ACR admin disabled, generated passwords
DevSecOps	TruffleHog (git history), Trivy image scan, Trivy IaC scan; HIGH/CRITICAL = pipeline fails	All GitHub Actions pinned to commit SHAs (lesson from the March 2026 trivy-action supply-chain incident)
CI/CD	GitHub Actions: Ruff + 17 pytest tests → build → scan → push → deploy → smoke test	OIDC federation; custom least-privilege role; runner IP added to AKS API allowlist only during deploy
Kubernetes	Deployment, LoadBalancer Service, ConfigMap, HPA (2-4 Pods), liveness/readiness probes, Key Vault CSI SecretProviderClass	App uses a restricted DB login; admin credentials never reach Pods
Observability	Prometheus + Grafana in AKS	Pod CPU, memory and network dashboards

3. Success criteria - evidence

3.1 Continuous Deployment. Changing the API title in *main.py* and pushing produced a green run; new Pods replaced the old ones by rolling update and the change was live on the public IP without downtime.

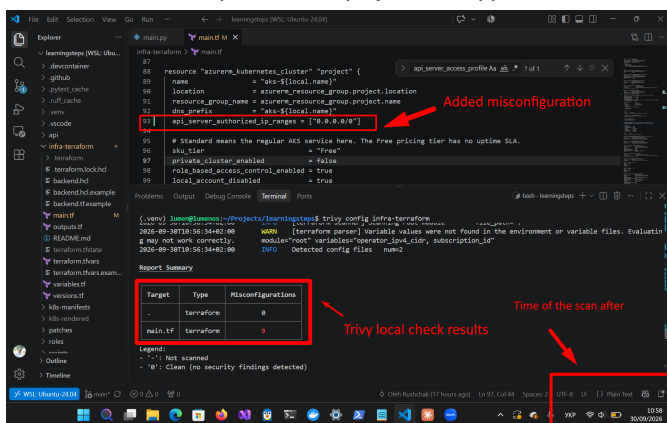
```
Problems Output Debug Console Terminal Ports
(.venv) lumen@lumenos:~/Projects/learningsteps$ git push
(.venv) lumen@lumenos:~/Projects/learningsteps$ git pull --rebase
Successfully rebased and updated refs/heads/main
(.venv) lumen@lumenos:~/Projects/learningsteps$ git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 439 bytes | 439.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/olehrush/learningsteps.git
 5e68de5..7145c5c main -> main
(.venv) lumen@lumenos:~/Projects/learningsteps$ kubectl -n learningsteps get pods --watch
NAME                                READY   STATUS    RESTARTS   AGE
learningsteps-65979cd648-6k72n      1/1     Running   0           75s
learningsteps-65979cd648-nx9ls      1/1     Running   0           67s
```

Push → new Pods (age 67-75 s) created by the pipeline

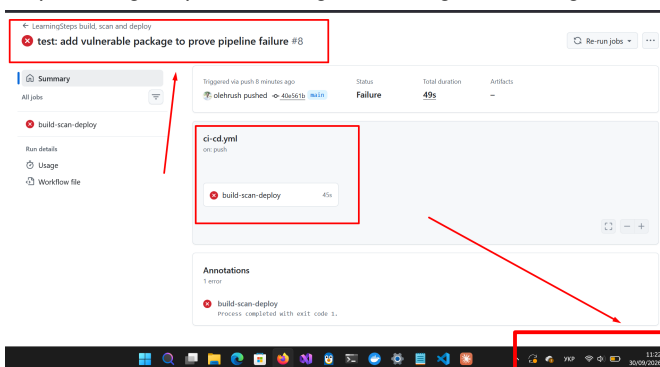


New title live on <http://57.168.17.48/docs>

3.2 Security Enforcement. Baseline IaC scan was clean (exit 0). An insecure rule (*api_server_authorized_ip_ranges* = ["0.0.0.0/0"], Kubernetes API open to the internet) was detected by Trivy (9 findings). A known-vulnerable dependency pushed to *main* failed run #8 with exit code 1; push and deploy were skipped and the live app kept running the previous image. Reverting restored a green run.



Insecure AKS API rule → Trivy IaC findings



Vulnerable package → pipeline run #8 fails, no deployment

3.3 Infrastructure Recovery. GitHub's deploy role was revoked first (its scope would disappear), then *terraform destroy* removed all 29 resources and *terraform apply* recreated them (~13 min). The state backend survived because it lives in a separate resource group. Cluster, DB schema, identities and pipeline variables were then re-initialised and the app deployed again.

```

main@lumenos:~/Projects/learningsteps$ terraform -chdir=infra-terraform apply destroy.tfplan
azure_rm_virtual_network_project Destroying... [id=/subscriptions/61707f9f-b6a8-4295-a11b-6136b305483/resourceGroups/rg-isoleah-1f1613/providers/Microsoft.Network/virtualNetworks/vnet-isoleah-1f1613]
azure_rm_subnet_aks Still destroying... [id=/subscriptions/61707f9f-b6a8-4295-a11b-6136b305483/resourceGroups/rg-isoleah-1f1613/providers/Microsoft.Network/subnets/subnet-aks]
azure_rm_subnet_aks Destruction complete after 11s
azure_rm_virtual_network_project Destroying... [id=/subscriptions/61707f9f-b6a8-4295-a11b-6136b305483/resourceGroups/rg-isoleah-1f1613/providers/Microsoft.Network/virtualNetworks/vnet-isoleah-1f1613]
azure_rm_virtual_network_project Still destroying... [id=/subscriptions/61707f9f-b6a8-4295-a11b-6136b305483/resourceGroups/rg-isoleah-1f1613/providers/Microsoft.Network/virtualNetworks/vnet-isoleah-1f1613]
azure_rm_virtual_network_project Destruction complete after 11s
azure_rm_resource_group_project Destroying... [id=/subscriptions/61707f9f-b6a8-4295-a11b-6136b305483/resourceGroups/rg-isoleah-1f1613]
azure_rm_resource_group_project Still destroying... [id=/subscriptions/61707f9f-b6a8-4295-a11b-6136b305483/resourceGroups/rg-isoleah-1f1613]
azure_rm_resource_group_project Destruction complete after 11s
random_string_suffix Destroying... [id=1f1613]
random_string_suffix Destruction complete after 0s
Apply complete! Resources: 0 added, 0 changed, 29 destroyed.
main@lumenos:~/Projects/learningsteps$

```

Destroy: 29 destroyed

```

main@lumenos:~/Projects/learningsteps$ terraform -chdir=infra-terraform apply apply.tfplan
with azure_federated_identity_credential_app
on main.tf line 315, in resource "azure_federated_identity_credential" "app":
315:   resource_group_name = azure_rm_resource_group.project.name
This field is no longer used and will be removed in the next major version of the Azure Provider
(and one more similar warning elsewhere)
Apply complete! Resources: 29 added, 0 changed, 0 destroyed.
Outputs:
acr_id = "/subscriptions/61707f9f-b6a8-4295-a11b-6136b305483/resourceGroups/rg-isoleah-1f1613/providers/Microsoft.ContainerRegistry/registries/acrisoleah1f1613"
acr_login_server = "acrisoleah1f1613.azurecr.io"
acr_name = "acrisoleah1f1613"

```

Recreate: 29 added (new suffix)

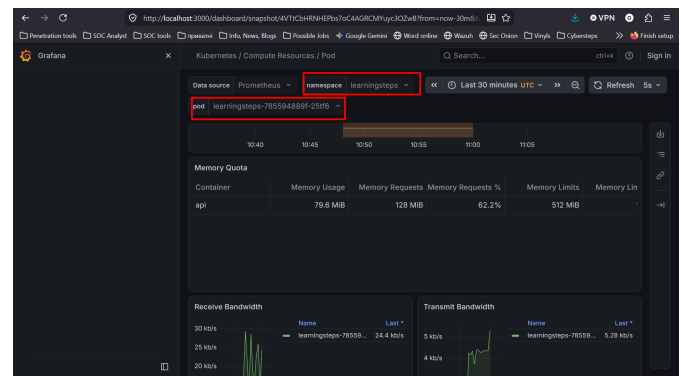
3.4 Self-healing and monitoring (bonus). A deleted Pod was replaced within seconds while the second replica kept serving traffic; data persisted in PostgreSQL. Grafana (kube-prometheus-stack) shows live Pod metrics for the *learningsteps* namespace.

```

lumen@lumenos:~/Projects/learningsteps$ kubectl -n learningsteps get pods
NAME                                READY   STATUS    RESTARTS   AGE
learningsteps-785594889f-25t46      1/1     Running   0           40m
learningsteps-785594889f-h28s8      1/1     Running   0           40m
lumen@lumenos:~/Projects/learningsteps$ kubectl -n learningsteps delete pod <learningsteps-785594889f-h28s8>
pod "learningsteps-785594889f-h28s8" deleted from learningsteps namespace
lumen@lumenos:~/Projects/learningsteps$ kubectl -n learningsteps get pods --watch
NAME                                READY   STATUS    RESTARTS   AGE
learningsteps-785594889f-25t46      1/1     Running   0           42m
learningsteps-785594889f-fv2zs      1/1     Running   0           10s

```

Pod deleted → replacement Running after 10 s



Grafana: Pod memory and network

4. Challenges and lessons learned

- Making the pipeline fail on purpose was harder than making it green.** Removing the AKS API allowlist only produced LOW/MEDIUM findings, so with *--severity HIGH,CRITICAL* Trivy returned exit 0 - the gate is only as strong as its severity threshold. A first vulnerable-package attempt (*requests==2.0.0*) also did not produce a usable failing case; a different known-vulnerable dependency was needed to prove the image gate.
- Getting to green first:** runs #1-#4 failed on secret-scan and image findings. Fixes: placeholder instead of a password-like DATABASE_URL in samples, TruffleHog limited to *verified* secrets (inherited false positives), Trivy *ignore-unfixed* so only fixable vulnerabilities block releases.
- Cloud constraints are real:** 10-vCPU quota per VM family (DSv5 unavailable, increase rejected) → D4s_v6; PostgreSQL restricted in Germany West Central → West Europe. No room for a surge node, so automatic AKS upgrades were disabled (documented accepted risk). azure_rm 4.81 does not support *max_unavailable* on the default pool - *terraform validate* caught it.
- Environment matters:** WSL2 Ubuntu instead of PowerShell (same OS as the CI runner), repo outside OneDrive so state and .env files never sync to the cloud, device-code login for az/gh in WSL.

5. Known limitations

- HTTP only, no API authentication - demo data only. Next step: Ingress + TLS (cert-manager) and auth.
- Recovery recreated infrastructure with a newly initialised database; journal data is not restored from backup.
- Manual Kubernetes version upgrades (quota leaves no surge capacity); OS patches applied in place (Unmanaged channel).
- ignore-unfixed* and verified-only secret scanning trade some coverage for fewer false positives; unfixed CVEs need separate tracking.